Q1: Consider the following code

```
add x10, x2, x3
add x11, x2, x5
sub x12, x10, x2
add x13, x12, x11
```

1. Identify *all the* instruction that have hazard(s).

Sol:

`sub x11` depends on `add x10`
`add x13` depends on `sub x12` and `add x11`

2. Re-write the code after inserting nops such that we don't have any hazards.

Sol:

```
add x10, x2, x3
add x11, x2, x5
nop
sub x12, x10, x2
nop
nop
add x13, x12, x11
```

3. Assume that register x1 initially contains the value 1, x2 contains 2, x3 contains 3, and so on. For each instruction, specify the pipeline stage it will be in during each clock cycle. Additionally, determine the final value of each register after the code executes on a pipelined processor that does not implement hardware hazard detection.

Sol:

	cc1	cc2	cc3	cc4	cc5	cc6	cc7	cc8
`add x10`	F	D	E	M	W			
`add x11`		F	D	E	M	W		
`sub x12`			F	D	E	M	W	
`add x13`								
`x10`	10	10	10	10	**5**	5	5	5
`x11`	11	11	11	11	11	**7**	7	7
`x12`	12	12	12	12	12	12	**8 (3)**	8
`x13`	13	13	13	13	13	13	13	**23(10)**